

CHAPTER - 4

IDENTIFYING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION

The Alibaba B2B E-Commerce Database Management System, built around the JBL Bluetooth earbuds wholesale sourcing case study, is designed to efficiently manage suppliers, categories, products, customers, orders, order items, payments, and reviews. Functional dependencies help identify how non-key attributes relate to the primary key in each table, while redundancy analysis and data anomaly detection ensure that data remains consistent, accurate, and free of unnecessary duplication. A well-designed database of this kind improves query performance, maintains referential integrity, and supports reliable sourcing and sales analysis for wholesale buyers.

4.2 SUPPLIER TABLE

Functional Dependency

Supplier_ID → Supplier_Name, Location, MOQ_Policy, Certification_Status, Contact_No, Email

Redundancy Analysis

Supplier details are stored only once in the Supplier table. Products, Order_Items, and Reviews reference the supplier using Supplier_ID instead of repeating supplier information.

Data Anomalies

- Insert Anomaly: A new supplier can be added even before listing any product.
- Update Anomaly: Updating a supplier's location or certification status requires a change only in the Supplier table.
- Delete Anomaly: Deleting a product listing does not remove the supplier's record.

4.2. CATEGORY TABLE

Functional Dependency

Category_ID → Category_Name, Description

Redundancy Analysis

Each category is stored only once. Products reference the category using Category_ID instead of repeating the category name and description.

Data Anomalies

- Insert Anomaly: New categories can be added without any product being listed under them.
- Update Anomaly: Renaming a category or editing its description is done only once.
- Delete Anomaly: Deleting a product does not delete the category.

4.3. PRODUCT TABLE

Functional Dependency

Product_ID → Product_Name, Category_ID, Supplier_ID, Price, MOQ, Listing_Type, Key_Features, Units_Sold

Redundancy Analysis

Product details are stored only in the Product table. Order_Item records use Product_ID instead of repeating product name, price, or features for every order.

Data Anomalies

- Insert Anomaly: A new product can be listed by a supplier before it receives any order.
- Update Anomaly: Changing a product's price or key features is updated only once in the Product table.
- Delete Anomaly: Deleting an Order_Item does not remove the product listing itself.

4.4. CUSTOMER TABLE

Functional Dependency

Customer_ID → Business_Name, Contact_No, Email, Country, Address

Redundancy Analysis

Customer/buyer information is stored only once in the Customer table. Orders and Reviews store only the Customer_ID, preventing duplicate customer records.

Data Anomalies

- Insert Anomaly: A new buyer can be registered without placing any order.
- Update Anomaly: Updating a customer's contact number or address requires a change only in the Customer table.
- Delete Anomaly: Deleting an order does not remove the customer's business record.

4.5. ORDER TABLE

Functional Dependency

Order_ID → Customer_ID, Order_Date, Order_Status, Total_Order_Value

Redundancy Analysis

The Order table stores only order-level information. Customer details are referenced using the Customer_ID foreign key rather than being repeated.

Data Anomalies

- Insert Anomaly: An order can only be created once a Customer_ID already exists, since every order must belong to a registered buyer.
- Update Anomaly: Order_Status can be updated (e.g., Pending to Shipped) without affecting customer or product data.
- Delete Anomaly: Deleting an order does not remove the customer record, but does affect its linked Order_Items and Payment if cascading is not handled carefully.

4.6. ORDER_ITEM TABLE

Functional Dependency

Order_Item_ID → Order_ID, Product_ID, Supplier_ID, Quantity, Unit_Price

Redundancy Analysis

This table connects Orders, Products, and Suppliers, avoiding repeated product or supplier information inside the Order table itself.

Data Anomalies

- Insert Anomaly: A product can be added as an order line item only if the Order, Product, and Supplier already exist.
- Update Anomaly: Changing the Quantity or Unit_Price of one item does not affect any other order item.
- Delete Anomaly: Removing an order item does not delete the underlying product or supplier record.

4.7. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Status, Payment_Date, Amount

Redundancy Analysis

Payment information is stored separately from the Order table, avoiding duplicate payment records for the same order (the One-to-One relationship ensures a single payment entry per order).

Data Anomalies

- Insert Anomaly: A payment record can be created only after the corresponding order exists.
- Update Anomaly: Payment_Status can be updated (e.g., Pending to Paid) without affecting the order or customer data.
- Delete Anomaly: Deleting a payment record does not remove the associated order, but leaves the order without proof of payment.

4.8. REVIEW TABLE

Functional Dependency

Review_ID → Customer_ID, Supplier_ID, Rating, Review_Text, Review_Date

Redundancy Analysis

Review information is stored separately from Customer and Supplier tables, avoiding repetition of business or contact details for every review submitted.

Data Anomalies

- **Insert Anomaly:** A review can be added only if both the Customer and Supplier already exist in their respective tables.
- **Update Anomaly:** Editing a review's rating or text affects only that specific review record.
- **Delete Anomaly:** Deleting a review does not remove the customer's or supplier's record.

4.9.CONCLUSION

The Alibaba B2B E-Commerce Database Management System is designed using proper functional dependencies and normalized tables for Suppliers, Categories, Products, Customers, Orders, Order_Items, Payments, and Reviews. Each entity stores only its relevant data, with foreign keys used to link related information instead of repeating it, thereby reducing redundancy and preventing insert, update, and delete anomalies. This design improves data consistency, maintains referential integrity, and ensures an efficient and reliable database for managing wholesale sourcing, pricing comparisons, and order tracking on a B2B platform.